

BJTU-RTS视觉算法组学习指南

撰写：赫晨皓 陈禹同 校对：陈禹同

前言

非常高兴你对北京交通大学RTS机器人队的视觉算法方面感兴趣，在这篇指南接下来的部分中，我们将会引导你提前熟悉并学习我们后续开发所需要使用到的一些工具以及开发环境。

说明

- 完成考核题请**不要进行任何形式的抄袭**。在面试时会针对考核题进行提问，因此各位务必保证是考核题为自己完成。当然可以参考，但是需要能够完整理解代码内容。
- 如果在完成考核的过程中有任何问题，请与组长私聊沟通，组长会帮助你进行考核，~~而且不用担心~~扣分。对于共性问题，组长将会将一部分聊天记录转到群中，希望大家能够理解。
- 关于生成式人工智能模型：我们**完全不限**大家对生成式模型的使用，并且鼓励大家逐渐摸索出一个适合自己的使用生成式模型的方法来提升自己的学习新知识的效率，以及降低debug时的红温程度。
- **所有考核所需的资料均在data.zip中**
- **关于文档**：考核中要求的文档均要求用markdown编写并转成pdf格式提交
- **关于考核题需要提交的内容**：将会在本指南末尾进行汇总，每部分的task将仅叙述任务内容

🌟好的，那么接下来就让我们开始这一次的学习之旅~

PART1：一切的开始：Linux

在后续的工作中，我们采用的是Ubuntu图形化界面，综合考量兼容性与易用性，我们具体采用的是Ubuntu20.04版本，对于该版本Linux系统的安装，有以下的方案可以实现：

- 方案一：虚拟机
 - [vmware安装Ubuntu](#)
- 方案二：双系统
 - [实体机安装双系统](#)

Tip1:

Linux分区的空间建议在100G以上，才会在后续的工作开展时不会显得捉襟见肘。当然，你也可以选择暂时先分配50G左右的空间给Linux用于完成本指南中的任务，后续再重新为Linux分配更大的空间。重装系统在视觉算法组也较为常见，因为在具体工作开展的过程中，环境爆炸是一个很难避免的问题，这也体现出Git的重要性

Tip2:

如果电脑的剩余空间已经无法分配足够的空间用于安装Linux，可以采用[外接硬盘的方式安装](#)

Task1: 初入命令行

可以参考MIT的经典课程的第一课，开始shell的学习和使用，并完成官网note的作业

- 官网note: [Topic 1: The Shell](#)
- 中字课程视频【中字】[The Missing Semester 第1讲](#)

Task2: 学习使用markdown

实际上本文档就是使用markdown工具撰写的，在这一个task中，各位的任务就是学习markdown文档的编写，Markdown是一种轻量级标记语言，它使用易读易写的纯文本格式编写文档，可与HTML混编，可导出 HTML、PDF 以及本身的 .md 格式的文件。不过千万不要被「标记」、「语言」吓到，Markdown的语法十分简单，常用的标记符号不超过十个，不到半小时就能完全掌握。

- [Markdown官方教程](#)
- 如何开始编写Markdown文档：
 - vscode插件: Markdown All in One
 - 一些所见即所得的软件，譬如: typora等
 - 文本编辑器 (记事本)

PART2: 平行时空展开! Git与GitHub

在这一部分的开始，让我们了解一下Git是什么?

Git是一个分布式版本控制系统，程序员管理代码的终极时间机器！旨在高效管理项目代码的版本和协作开发。

Git 的核心能力

1. 版本控制:

- 随时回到代码的任意历史版本（即使昨天删错文件也不怕）
- 查看“谁在何时改了哪行代码”（`git blame`）

2. 分支管理:

- 主分支（main分支）稳定运行
- 分身出多个分支: `feature/login`（开发新功能）、`hotfix`（紧急修bug）

3. 分布式协作:

- 团队成员各自拥有完整代码副本
- 通过 `push` / `pull` 同步变化，自动解决冲突

经典场景演示

```
# 1. 初始化仓库
git init

# 2. 创建当前版本快照
git add .
git commit -m "实现了登录功能"

# 3. 创造分支
git checkout -b experiment

# 4. 发现错误？回到过去！
git reset --hard 版本号
```

为什么开发者离不开它？

- **离线工作**：没有网络也能提交代码（vs SVN）
- **绝对安全**：每个人的电脑都是完整备份
- **生态强大**：GitHub/GitLab/Bitbucket 的基石

一句话灵魂比喻：

"Git 可以让你在代码世界中自由跳跃！"

Task3: 创建Git仓库

在目前的工作中，我们组内的大量手册以及后续项目的具体算法都存放在github的某个仓库中。那么在这一部分中，需要大家学习使用git，将此考核文件git到你自己的电脑上就是本指南中的一项考核内容，你可以使用vscode插件或者jetbrains旗下的软件自带的git功能，也可以使用git的命令行工具来完成，这里只提供git的命令行工具，能够自己寻找到适合自己的教程也是能力考核的一部分。下面是一些参考资料：

- [博客园：Git基础使用教程](#)
- [Git快速入门](#)
- [missing-semester: 版本控制 \(Git\)](#)

PART3：编译，全速编译！cmake

在这一部分的开始，让我们了解一下CMake 是什么？

想象你是一个包工头（开发者），要盖一栋楼（C/C++项目）。但工地上有：

- **不同施工队**（Windows/MSVC、Linux/GCC、Mac/Clang...）
- **各种建材**（源码、库文件、头文件...）
- **奇葩需求**（Debug版、Release版、动态库、静态库...）

如果每次换施工队都要手写一套施工图纸（Makefile/VS项目），你会疯掉！

这时，**CMake 闪亮登场** ✨：

1. 你只需写一份“通用蓝图”（CMakeLists.txt），用简单的语法声明：

- “楼有几层”（项目结构）
- “用啥钢筋水泥”（依赖库）
- “要不要装电梯”（编译选项）

2. CMake 自动生成对应施工队的专用图纸：

- 对 Linux 生成 `Makefile`
- 对 Windows 生成 `.sln` (Visual Studio 项目)

3. 甚至帮你喊卡车运建材 (自动下载依赖库, 比如通过 `FetchContent` 或 `find_package`) 。

举个栗子：

```
cmake_minimum_required(VERSION 3.10) # 声明CMake版本
project(MyAwesomeApp)                # 项目名

add_executable(hello main.cpp)        # 用main.cpp建一个可执行文件hello
target_compile_features(hello PRIVATE cxx_std_11) # 要求C++11
```

然后运行：

```
mkdir build && cd build
cmake .. # 生成本地构建系统
make     # 开盖即用 (Linux/Mac)
```

总之，CMake 是 C/C++ 世界的“万能翻译官”+“自动化施工队”！

Task4：学习使用cmake

参考以下的内容初步学习cmake的作用，完成基本的CmakeLists.txt的编写以及Ubuntu系统下的项目编译

- [软件构建: CMake 快速入门](#)
- [ubuntu安装 cmake , 以及使用](#)

PART4：视觉魔法启动！OpenCV

openCV是什么，能完成什么工作，和opencv是亲戚吗

OpenCV (open source computer vision library) 是一个基于BSD许可 (开源) 发行的跨平台计算机视觉库，可以运行在Linux、Windows、Android和Mac OS操作系统上。

它轻量级而且高效——由一系列 C 函数和少量 C++ 类构成，同时提供了Python、Ruby、MATLAB 等语言的接口，实现了图像处理和计算机视觉方面的很多通用算法。

OpenCV用C++语言编写，它的主要接口也是C++语言，但是依然保留了大量的C语言接口。

在计算机视觉项目的开发中，OpenCV作为较大众的开源库，拥有了丰富的常用图像处理函数库，采用C/C++语言编写，可以运行在Linux/Windows/Mac等操作系统上，能够快速的实现一些图像处理和识别的任务。

此外，OpenCV还提供了Java、python、cuda等的使用接口、机器学习的基础算法调用，从而使得图像处理和图像分析变得更加易于上手，让开发人员更多的精力花在算法的设计上。

核心功能 (1) 图像处理****

- **基础操作：**读取、保存、裁剪、旋转、缩放、颜色空间转换
- **滤波与增强：**
 - 平滑 (高斯模糊、中值滤波)
 - 锐化 (边缘增强)
 - 去噪 (非局部均值去噪)

- **形态学操作**：腐蚀、膨胀、开运算、闭运算（用于二值图像处理）
- **直方图均衡化**：改善图像对比度

(2) 特征检测与匹配

- **关键点检测**：
 - SIFT、SURF（尺度不变特征）
 - ORB、FAST（实时特征检测）
- **特征匹配**：BFMatcher、FLANN 匹配算法
- **角点检测**：Harris 角点、Shi-Tomasi 角点

.....

优势

跨平台：支持 Windows/Linux/macOS/Android/iOS

多语言支持：C++、Python、Java、MATLAB

高性能：底层优化（C++/CUDA 加速）

丰富的预训练模型：可直接调用经典 CV 模型

Task5：学习使用源码编译安装opencv (C++)

再这一个task中，我们需要你通过源码编译并安装opencv，版本不限。并在安装完成后编写一个小的项目，通过opencv的库函数，结合cmake编译生成一个可执行文件来随意显示一张自己喜欢的图片。下面是一些参考资料：

- [Ubuntu搭建OpenCV环境](#)
- [opencv教程](#)

PART5：核心启动！ROS（机器人操作系统）

ROS（Robot Operating System，机器人操作系统）是一个开源的机器人开发框架，提供了一系列工具、库和约定，用于简化机器人软件的开发。虽然名为“操作系统”，但它并非传统意义上的OS（如Windows/Linux），而是运行在现有操作系统（如Ubuntu）之上的中间件。

ROS的核心特点

模块化设计

采用节点（Node）通信机制，每个功能模块可独立开发、测试和部署。

节点间通过话题（Topic）、服务（Service）、动作（Action）进行数据交互。

丰富的工具集

Rviz：3D可视化工具，实时显示传感器数据、机器人模型等。

Gazebo：高保真物理仿真环境，支持机器人运动和控制测试。

rqt：图形化调试工具（如绘制数据曲线、动态参数调整）。

跨语言支持

支持 C++ 和 Python（主流开发语言），社区提供其他语言接口（如Java、LISP）。

庞大的生态系统

官方提供数千个开源包（如导航SLAM、机械臂控制）。

支持主流传感器（激光雷达、摄像头、IMU）和机器人硬件（TurtleBot、UR机械臂）。

版本演进

- ROS 1 (Noetic) : 经典版本,成熟稳定,我们目前主要使用这一版本
- ROS 2 (Humble/Foxy) : 支持DDS通信协议,提升实时性和跨平台能力

推荐教程

[B站:机器人工匠阿杰](#)

Task6: 编写一个简单的ROS功能包

在这一个Task中,你们需要在自己的电脑上完成如下的任务:

- 使用鱼香ros工具更换系统源并安装ros1 (目前队内使用的是ros1)
- 使用话题通信, 完成一个发布和订阅的工作, 其中发布的是三个数, 订阅同理, 订阅后将收到的三个数显示在屏幕上, 以这样的格式 (数字无所谓)

```
x:111 y:111 z:232
```

- 对于这个任务还有如下的限制:
- 需要自定义msg的消息类型:为三个int64
- 需要使用launch文件启动发送与接受节点,而不是roslaunch
- 使用cpp或python实现(优先使用cpp编写)

PART6:一点点人工智能-YOLOv8

尝试自己运行一个深度学习模型!

搭建深度学习环境

Task7: 制作一个数据集并训练一个yolo模型

参考网上的各种教程,采用我们提供的数据集,自行标注数据集,标注数据按照数据集中的data.yaml文件中的顺序进行标注,训练一次yolov8模型[参考资料](#)

- [标注工具](#)

PART7:最后的最后-来一锅大杂烩!

通过之前的学习,相信你已经对视觉算法组的工作有了一个初步的了解,那么在接下来就让我们把之前的所学进行综合,完成一个相对复杂一些的任务!

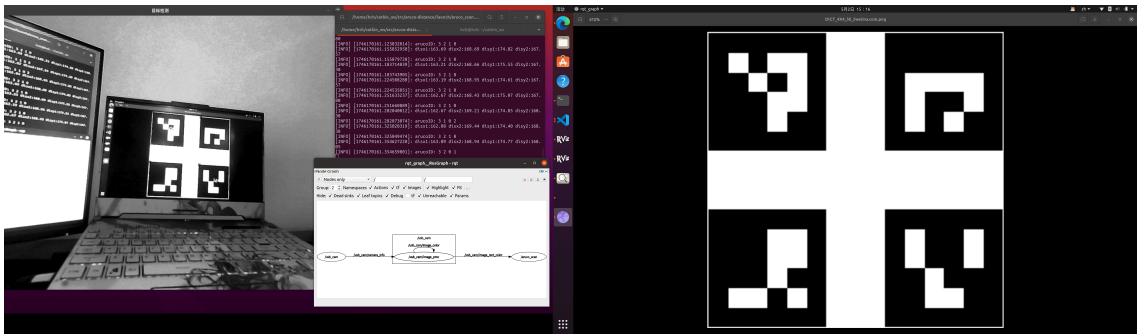
在这部分任务中,我们在不会向大家提供过于详实的资料,在完成任务的过程中需要大家自行搜集查阅资料并解决问题,但是如果在遇到难以解决的问题依然可以与组长联系并获得帮助,毕竟摇人也是一种获取资料和帮助的渠道! 不会扣分,但是还是要尽量独立去完成这一部分的任务,因为可能会遇到一些初学者难以定位到问题根源的bug,一味追求独立解决可能会绕弯路进而浪费大量时间

Task8: 最终挑战

该部分任务为可选项,但是完成后有额外加分

固定任务:

- 在第一个ros功能包中使用我们提供的rosv8数据流, 订阅其中录制的相机传回的彩色视频流节点, 识别彩色视频流中的aruco标识码, 将识别到的aruco标识的ID发布到roscore中, 使用ros的命令行工具将一些信息可视化展现, 类似于这个效果:



在这张图片中，我向终端输出了aruco码的ID以及每相邻两个aruco中心点的距离（像素），基于这些数据结合焦距的等信息后，就可以计算出被识别平面的角度和距离

- 使用rqt、rviz等可视化工具，将彩色视频流中的图像可视化

自由发挥：

- 在另一个ros功能包中，进行一些**自由发挥**，比如可以通过订阅统计出现过的aruco标识，亦或是利用单目测距的原理结合相机焦距等信息对aruco标识进行测距，亦或是通过可视化工具可视化更多的信息。
- 总之，在这一部分不做任何限制，**一切随心、量力而为**。毕竟考试要紧（手动狗头）

Info

这部分所使用的rosvbag包由于GitHub文件体积限制，故无法上传至GitHub仓库，获取方式及aruco标识（例子）均放到了aruco-data文件夹下

重要信息：每Task任务提交清单汇总

- 个人简历（模板在微信群和仓库中均有）
- Task1：
 - 一个文档：官网作业命令行任务在电脑运行截图，以及填写的对应该命令行的理解
- Task4 & Task 5：
 - 编写的代码文件
 - cmakeLists文件
 - 运行效果截图
 - 一个文档详细解释你的文件和实现过程
- Task6：
 - 发送消息的功能包
 - 接受消息的功能包
 - 一个解释文档

其中一个功能包需包含自定义msg消息文件

- Task7：
 - 训练完成生成的pt文件
 - 识别效果截图(results.png)
 - 电脑上运行截图
 - 一个解释文档
- Task8：

- 编写的所有功能包
- 运行时截图
- 一个解释文档（描述这两个包都实现了什么功能）

Important!

所有文件的提交均需要提交到GitHub的个人仓库，并将仓库链接提交到微信群中腾讯文档中。

试用水印